

AirPrice India -- SIH26056 Prototype Documentation

Smart Hackathon India 2026 | Problem ID: SIH26056 | Status: Live and Deployed

1. What Is This Project?

The Simple Problem

Every month, India's Ministry of Statistics (MoSPI) publishes the Consumer Price Index (CPI). But MoSPI's airfare data is collected manually and published with a 30-45 day delay. By the time you read it, the price has already changed.

Our Solution

We built a fully automated system that scrapes live airfare prices from the internet every day, calculates its own real-time airfare CPI, and shows it on an interactive dashboard -- giving a signal that is 30 days FASTER than the official government data.

In One Line

We are the real-time airfare equivalent of the stock market ticker, but for CPI.

2. Technology Stack

Technology	Role in the Project
Python 3.11	Main programming language for all scripts.
fast-flights	Reads Google Flights live prices without a browser. currency=INR enforced explicitly.
Pandas	Data cleaning, outlier detection, CPI calculation.
Supabase / PostgreSQL	Cloud database. All fare data and computed CPI stored permanently.
Streamlit	Turns Python into a web dashboard accessible from any browser.
Plotly	Interactive India route map and color-coded CPI bar chart.
GitHub Actions	Free cloud automation. Runs pipeline 5x/day on Ubuntu servers even when PC is off.
scikit-learn	Isolation Forest ML algorithm for optional anomaly detection.
FastAPI + Uvicorn	Local mock airline API server for Tier 2 data collection.
tenacity	Auto-retry with exponential backoff for failed network requests.
python-dotenv	Loads database credentials from .env file safely.
schedule (Python)	Triggers the local pipeline at specific clock times.
SQLAlchemy	Connects Python code to the PostgreSQL database.

3. Complete Data Flow

Step 1: Trigger (5 times per day)

smart_scheduler.py on your PC or GitHub Actions cron on a cloud server starts the pipeline.

Step 2: Data Collection

collector_tier1.py queries Google Flights for economy fares on 3 routes (DEL-BOM, BOM-BLR, DEL-BLR) for 4 booking windows (3, 7, 14, 30 days ahead). Raw prices are written to the fare_observations table in Supabase.

Step 3: Data Cleaning

cleaning.py reads all raw rows. Duplicates are flagged (same source + route + airline + hour). Outliers are flagged using the IQR method (prices outside 1.5x the normal range). Data is NEVER deleted -- only flagged. All rows written to fare_observations_clean.

Step 4: CPI Calculation

index_calc.py reads only clean rows (is_duplicate=FALSE, is_outlier=FALSE). For each route, computes: $CPI = 100 \times (\text{Current Avg Price}) / (\text{Base Period Avg Price})$. Results written to the airfare_index table.

Step 5: Anomaly Detection

anomaly.py computes a rolling Z-score per route over a 10-observation window. If any price has a Z-score > 2.5, an ALERT is logged for human review.

Step 6: Dashboard

dashboard.py reads all data live from Supabase on every page load and renders: interactive India route map, fare trend charts, CPI metrics, and MoSPI benchmark comparison.

4. The CPI Formula

Route-Level CPI

$CPI = 100 \times (\text{Average Price Today}) / (\text{Average Price on Base Date})$

Example: Base date fare Rs 5,000 (CPI=100). Current fare Rs 5,500. $CPI = 100 \times 5500/5000 = 110$. Meaning: 10% INFLATION since the base period.

National Airfare CPI (All Routes Combined)

National CPI = $\text{Sum}(\text{Route CPI} \times \text{Observation Count for Route}) / \text{Total Observations}$. Routes with more observations (more passengers) have more weight. This is the Laspeyres Index formula -- the same formula used by India's official CPI.

Festival Spikes: Include or Exclude?

INCLUDE. A Diwali price of Rs 15,000 when the average is Rs 7,000 is REAL inflation -- the consumer actually paid it. CPI must reflect that burden. We only EXCLUDE data ERRORS: glitches showing Rs 9,99,999 fares, zero-rupee fares, or prices accidentally captured in USD instead of INR.

5. Automation Schedule

Target Time (IST)	Local Trigger	Cloud Trigger	Why This Time
04:00 AM	03:55 AM	03:30 AM (UTC 22:00)	Overnight pricing algorithms finish. New prices are live.

12:00 PM	11:55 AM	11:30 AM (UTC 06:00)	Midday repricing after morning booking demand.
07:00 PM	06:55 PM	06:30 PM (UTC 13:00)	Peak evening. Highest consumer activity. Prices often at maximum.
08:00 PM	07:55 PM	07:30 PM (UTC 14:00)	Secondary airline pricing update window.
11:00 PM	10:55 PM	10:30 PM (UTC 17:00)	Late-night clearance pricing before midnight system reset.

Note: Cloud times are 30 minutes earlier to account for GitHub free-tier cron queue delays (15-30 min). Local times are 5 minutes early because the scraper takes ~5 minutes to complete.

6. Key Problems Solved

Problem 1: USD vs INR Currency Bug [CRITICAL]

What happened: All database prices were 2-digit numbers (e.g., 93 INR) instead of thousands of rupees. CPI was completely wrong. Root Cause: GitHub Actions runs on US servers. Google detects the US IP and returns prices in USD (\$93). Our code saved this as 93 INR. Fix: Added currency='INR' explicitly to the Google Flights query. Deleted ~8,600 corrupt rows from the Supabase database.

Problem 2: Plotly 7.0 Deprecation [MEDIUM]

What happened: Dashboard crashed on startup with AttributeError: Scattermapbox. Root Cause: Plotly 7.0 completely removed the old Scattermapbox class and mapbox layout. Fix: Rewrote map code using the new Scattermap API and switched to OpenStreetMap tiles (no API key required).

Problem 3: Scheduler Not Reloading New Times [MEDIUM]

What happened: After adding 8 PM and 11 PM to the schedule, they never triggered. Root Cause: Python daemon processes do not auto-reload when source code changes. Fix: Always kill and restart the smart_scheduler.py process whenever the schedule code is modified.

Problem 4: GitHub Actions Cron Queue Delay [MEDIUM]

What happened: 8 PM scrape was arriving at 8:22 PM due to GitHub shared server queues. Root Cause: GitHub free-tier cron shares infrastructure with millions of developers. Fix: Shifted all cloud cron times 30 minutes earlier so data arrives by the target hour.

Problem 5: Deployed App Not Visible to Teammates [MINOR]

What happened: Teammates got 'You do not have access to this app' message. Root Cause: Private GitHub repo causes Streamlit to make the deployed app private by default. Fix: Changed app visibility to Public in Streamlit's sharing settings. Code stays private, dashboard is public.

7. Glossary of Terms

Term	Plain Language Meaning
CPI	Consumer Price Index. A number showing price changes. CPI=110 means 10% more expensive than base.
Base Period	The starting date. CPI = exactly 100 on this date.
Inflation	Prices going up. CPI > 100.
Deflation	Prices going down. CPI < 100.
Route	A flight path. DEL to BOM = Delhi to Mumbai.

Booking Window	Days in advance the ticket is purchased. 7-day = flight is 7 days from today.
Scraping	Computer program reading website data automatically at high speed.
Laspeyres Index	The official CPI formula used by India government. We use the same formula.
Outlier	A price so extreme it cannot be real. Flagged and excluded from calculations.
Anomaly	A suspicious price spike. Could be real (festival) or a bug. Humans investigate.
Z-Score	How unusual a number is vs recent average. $Z > 2.5$ = very unusual, alert fired.
Supabase	Cloud database service. All our data lives here permanently.
GitHub Actions	Free cloud server that runs our scraping code automatically 5 times a day.
Cron	A scheduling system for code. Like an alarm clock for computers.
Streamlit	Python library that automatically creates web apps from Python code.
IST / UTC	IST = Indian Standard Time (UTC+5:30). GitHub uses UTC, so we convert all times.
fast-flights	Python library that reads Google Flights data without opening a browser.
Venv (.venv)	Isolated Python environment with only this project's packages installed.
Tier	Data source ranking. Tier 1 = real scraped data. Tier 0 = government data.
Pipeline	Sequence of programs running one after another like a factory assembly line.